## ⭐ Relationships in Entity Framework Core

In a relational database, tables are connected through **foreign keys**.
In EF Core, the same connections are called **entity relationships**.

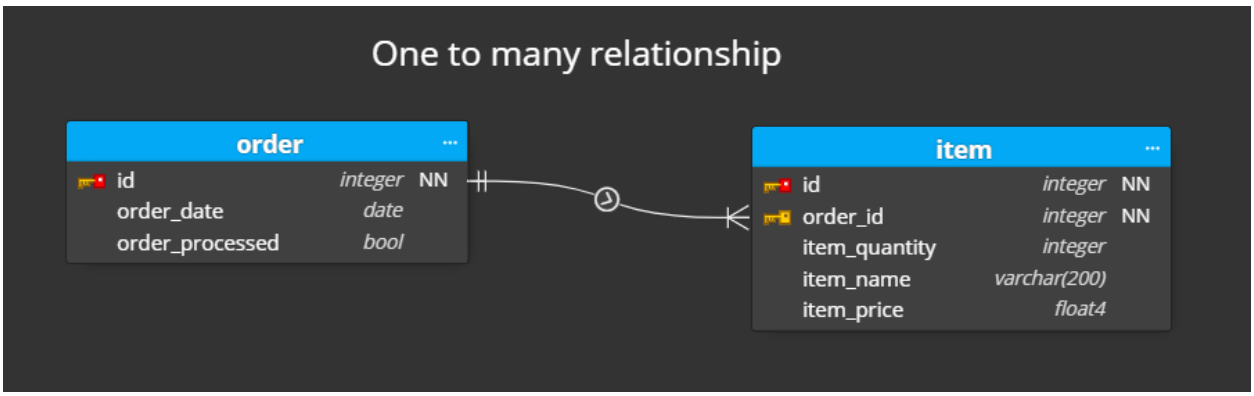
EF Core supports **three major types** of relationships:

1. **One-to-One (1:1)**
2. **One-to-Many (1:N)**
3. **Many-to-Many (M:N)**

---

## 1️⃣ One-to-Many Relationship (MOST COMMON)

➡️ Example: **One Category → Many Products**




### ✓ Concept

- One parent entity has multiple child entities
- Child entity contains the **foreign key**

### ✓ Example Classes

```
public class Category
{
    public int Id { get; set; }
    public string Name { get; set; }

    // Navigation Property
```

```
    public ICollection<Product> Products { get; set; } = new List<Product>();
}

public class Product
{
    public int Id { get; set; }
    public string Name { get; set; }

    // Foreign Key
    public int CategoryId { get; set; }

    // Navigation Property
    public Category Category { get; set; }
}
```

**✓ Fluent API**

```
modelBuilder.Entity<Product>()
    .HasOne(p => p.Category)
    .WithMany(c => c.Products)
    .HasForeignKey(p => p.CategoryId);
```

---

**2️⃣ One-to-One Relationship (1:1)**

➡️ Example: **User → UserProfile**

**✓ Concept**

- One entity has *exactly* one related entity
- One of the tables must contain the **foreign key with a unique constraint**

**✓ Example Classes**

```
public class User
{
    public int Id { get; set; }
    public string Username { get; set; }

    public UserProfile Profile { get; set; }
}

public class UserProfile
{
    public int Id { get; set; }

    // Foreign Key (also Primary Key)
    public int UserId { get; set; }

    public string Address { get; set; }

    public User User { get; set; }
}
```

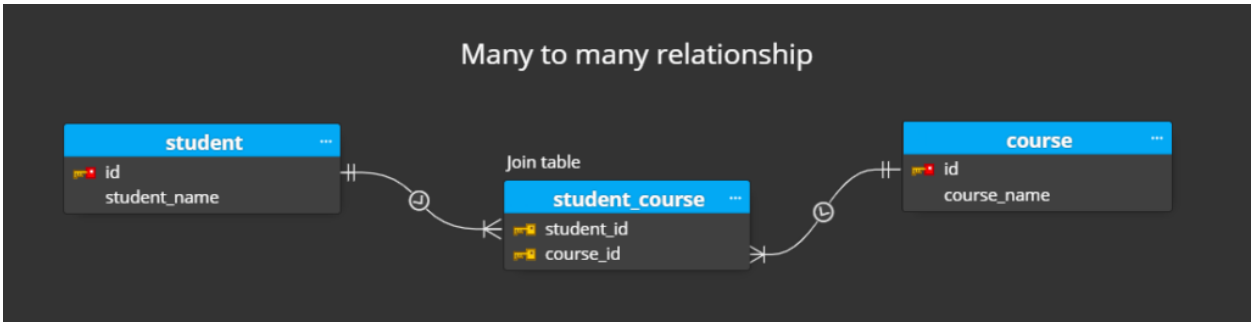
**✓ Fluent API**

```
modelBuilder.Entity<User>()
    .HasOne(u => u.Profile)
    .WithOne(p => p.User)
    .HasForeignKey<UserProfile>(p => p.UserId);
```

---

### 3️⃣ Many-to-Many Relationship (M:N)

➡️ Example: **Students ↔ Courses**




✓ **Concept**

- One entity can have many of another entity and vice versa
- EF Core automatically creates the **join table** from EF Core 5+

✓ **Example Classes**

```
public class Student
{
    public int Id { get; set; }
    public string Name { get; set; }

    public ICollection<Course> Courses { get; set; } = new List<Course>();
}

public class Course
{
    public int Id { get; set; }
    public string Title { get; set; }

    public ICollection<Student> Students { get; set; } = new List<Student>();
}
```

✓ **Fluent API** ***(Optional – EF Core can configure automatically)***

```
modelBuilder.Entity<Student>()
    .HasMany(s => s.Courses)
    .WithMany(c => c.Students)
    .UsingEntity(j => j.ToTable("StudentCourses"));
```

---

⭐ **Navigation Properties**

Navigation properties create object-level relationships:

| **Type** | **Navigation Example** |
|---|---|
| **Reference** | Product.Category |
| **Collection** | Category.Products |

---

⭐ **Foreign Keys (FK) in EF Core**

**Explicit FK**

```
public int CategoryId { get; set; }
```

**Shadow FK**

EF Core creates the FK even if you don't define it.

---

⭐ **Relationship Conventions (What EF Core detects automatically)**

**One-to-many detection:**

- If class A has collection of B → EF Core assumes one-to-many

**One-to-one detection:**

- If both sides have reference navigation → EF tries to configure 1:1

**Many-to-many detection:**

- If both sides are collections → EF creates join table automatically

---

### ⭐ Cascading Behaviors

When deleting a parent entity:

| Cascade Type | Effect |
|---|---|
| **Cascade** | Child rows deleted |
| **Restrict** | Prevent deletion if children exist |
| **SetNull** | FK becomes null |
| **NoAction** | Database decides |

Example:

```
modelBuilder.Entity<Category>()
   .HasMany(c => c.Products)
   .WithOne(p => p.Category)
   .OnDelete(DeleteBehavior.Cascade);
```

---

### ⭐ When to Use Each Relationship Type

| Relationship | Use Case |
|---|---|
| **One-to-One** | Additional details table (User ↔ Profile) |
| **One-to-Many** | Parent–child models (Category → Products) |
| **Many-to-Many** | Linking entities (Students ↔ Courses) |